

Day 1 종합실습 - 코드 분석 결과

데이터 수집 미니 파이프라인 | 광주캠퍼스 4반 권경현

1. 프로젝트 개요

공개 API 3개(Open-Meteo, Countries.dev, ip-api)를 asyncio와 httpx로 동시에 수집하고, Pydantic v2로 타입과 범위를 검증한 뒤, 검증을 통과한 데이터를 CSV와 Parquet 두 형식으로 저장하며 읽기/쓰기 성능을 비교하는 파이프라인이다. 단일 파일(main.py)로 구성하고 requirements.txt로 의존성을 고정했으며, pytest 단위 테스트와 ruff 코드 검사를 통과하고 git으로 이력을 관리했다.

Text

2. 코드 분석 결과에 대한 본인 의견

2-1. 파이프라인 설계

수집 - 추출 - 검증 - 저장의 네 단계를 각각 독립 함수로 나눠, 한 단계의 변경이 다른 단계에 영향을 주지 않도록 했다. 특히 검증 단계에서 통과분(valid)과 실패분(errors)을 나누는 구조 덕분에, 한 레코드가 잘못돼도 전체가 멈추지 않고 나머지를 살릴 수 있다. 실무 파이프라인에서 중요한 부분 실패 격리를 작은 규모로 구현한 셈이다.

2-2. 비동기 수집

asyncio.gather로 3개 요청을 동시에 실행해, 순차 호출이면 세 번의 대기가 합쳐질 것을 가장 느린 한 번의 대기 수준으로 줄였다. 또 각 요청을 감싼 fetch가 예외를 값(error)으로 바꿔 반환하므로, 한 API가 실패해도 gather 전체가 깨지지 않는다.

2-3. 스키마 검증

Pydantic 모델에 타입뿐 아니라 범위 제약(기온 -60~60, 강수확률 0~100, 위도 -90~90, 경도 -180~180)을 함께 걸어, 형식은 맞지만 값이 비정상인 데이터까지 걸러지게 했다. 검증 규칙이 코드가 아니라 스키마로 드러나 읽기 쉽다.

2-4. 성능 측정 - 가장 중요한 분석

처음 main.py를 실행했을 때(캡처 1) 72행밖에 안 되는데도 Parquet 쓰기가 43.9ms, 읽기가 70.5ms로 CSV(0.3ms대)보다 100배 이상 느리게 나왔다. 강의에서 배운 'Parquet이 CSV보다 빠르다'와 정반대라, benchmark.py(캡처 2)로 규모를 키워보고 profile_perf.py(캡처 3)와 memory_profiler(캡처 4)로 교차 검증했다.

- cProfile 결과 병목은 pandas의 to_csv(_save_chunk)였고, Parquet 자체의 실제 비용은 1ms 미만이었다.
- 즉 main.py의 43ms는 pyarrow의 최초 1회 초기화 비용이 timeit 50회 평균에 섞여 부풀려진 측정 아티팩트였다.
- benchmark.py에서는 약 1만 행을 기점으로 Parquet이 CSV를 역전했고, 100만 행에서는 읽기가 약 20배 빠르고 파일도 4배 작았다.

규모별 읽기 시간 (benchmark.py 실측)

행 수	CSV 읽기	Parquet 읽기
1,000	0.6 ms	9.8 ms
10,000	2.1 ms	0.6 ms (역전)
100,000	19.1 ms	1.5 ms
1,000,000	210.3 ms	10.2 ms

측정값을 곧이곧대로 믿지 않고 여러 도구로 교차 검증해 원인을 찾은 것이 이번 실습에서 가장 큰 배움이었다.

3. 추가 의견 - 개선 사항 (리팩토링)

3-1. 실제 적용한 개선

(1) warm-up 추가: 측정 직전에 각 형식을 한 번씩 미리 쓰고 읽어, pyarrow 초기화 같은 일회성 비용을 측정에서 제외했다. 그 결과 Parquet 측정치가 정상화됐다.

항목	개선 전	개선 후
Parquet 쓰기	43.879 ms	0.227 ms
Parquet 읽기	70.469 ms	0.330 ms
CSV 쓰기/읽기	0.382 / 0.243 ms	0.154 / 0.155 ms

(2) 네트워크 재시도: httpx 전송 계층에 재시도 2회를 붙여 일시적 연결 오류에 견디게 했다. 강의 파트 3의 재시도 개념을 적용한 것이다.

3-2. 향후 개선 제안

- 수집 데이터가 커지면 저장 기본 형식을 Parquet으로 두는 편이 시간과 용량 모두 유리하다(위 벤치마크가 근거).
- 현재는 서울 한 지점과 단건(국가/IP)이라 저장/성능 비교의 의미가 제한적이다. 여러 도시로 확장하면 파이프라인의 가치가 더 커진다.
- country/ip API 실패 시 지금은 검증에서 조용히 걸리는데, 실패를 명시적으로 로깅하거나 재시도하면 관측성이 좋아진다.
- 성능 비교의 반복 횟수(repeat=50)를 설정 값으로 빼면 대용량에서도 유연하게 조절할 수 있다.

4. 코드 품질 개선 측면

- 정적 검사: ruff(E, F, I, UP 규칙)로 스타일, 임포트 정렬, 문법을 일관되게 유지하고 검사를 통과했다(캡처 6).
- 테스트: pytest로 스키마의 정상/경계/오류 케이스 6건을 자동 검증했다(캡처 5). fixture와 parametrize로 중복을 줄였다.
- 주석과 가독성: 각 단계에 무엇을 왜 하는지와 관련 강의 파트를 표기해, 처음 보는 사람도 흐름을 따라갈 수 있게 했다.
- 재현성: requirements.txt로 버전을 고정하고, 경로를 __file__ 기준으로 잡아 폴더를 옮겨도 동작하며, git 커밋으로 이력을 남겼다.
- 로깅: print 대신 logger를 쓰고 httpx의 요청 로그는 낮춰, 출력이 파이프라인 흐름 위주로 깔끔하다.

5. 결론

규모는 작지만 비동기 수집, 선언적 검증, 성능 측정과 프로파일링, 단위 테스트, 정적 검사, 버전 관리까지 실무 데이터 파이프라인의 핵심 요소를 한 번에 경험했다. 무엇보다 성능 수치를 맹신하지 않고 cProfile, memory_profiler, 규모별 벤치마크로 교차 검증해 측정 아티팩트를 발견하고 코드를 고친 과정이, 단순히 돌아가는 코드를 넘어 믿을 수 있는 코드로 가는 연습이었다.

6. 산출물 캡처

캡처 1. python main.py 실행 - 3개 API 비동기 수집, 검증 72/72, CSV/Parquet 저장과 성능 비교

```
===== 8 passed in 11.79s =====
[ (.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 % ls
__pycache__      main.py          README.md        requirements.txt  test_main.py ]
[ (.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 % python main.py ]
INFO | 수집 시작 : 3개 API 동시 호출
INFO | weather 응답 : 정상
INFO | country 응답 : 정상
INFO | ip 응답 : 정상
INFO | 검증 결과 (valid/total): weather 72/72, country 1/1, ip 1/1
INFO | 저장 완료 : weather 72행 -> CSV, Parquet

[저장/성능 비교] (평균, 72행 기준)
쓰기 CSV 0.382 ms | Parquet 43.879 ms
읽기 CSV 0.243 ms | Parquet 70.469 ms
크기 CSV 1852 B | Parquet 3439 B

[국가/IP 요약]
country: {'name': 'Korea (Republic of)', 'region': 'Asia', 'population': 51780579, 'area': 100210.0, 'code': 'KOR'}
ip: {'ip': '8.8.8.8', 'country': 'United States', 'country_code': 'US', 'lat': 39.03, 'lon': -77.5, 'timezone': 'America/New_York'}

검증 실패 총 0건
[ (.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 % ]
```

캡처 2. benchmark.py - 규모별 CSV vs Parquet. 약 1만 행부터 Parquet이 역전한다

```
[ (.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 % python benchmark.py ]
규모별 CSV vs Parquet 성능 (쓰기/읽기 ms, 파일 MB)

[1,000행]
쓰기 : CSV      1.1 ms | Parquet    8.2 ms
읽기 : CSV      0.6 ms | Parquet    9.8 ms
크기 : CSV      0.03 MB | Parquet    0.01 MB

[10,000행]
쓰기 : CSV      4.5 ms | Parquet    1.1 ms
읽기 : CSV      2.1 ms | Parquet    0.6 ms
크기 : CSV      0.28 MB | Parquet    0.09 MB

[100,000행]
쓰기 : CSV     43.7 ms | Parquet     5.4 ms
읽기 : CSV     19.1 ms | Parquet     1.5 ms
크기 : CSV     2.79 MB | Parquet     0.73 MB

[1,000,000행]
쓰기 : CSV    437.8 ms | Parquet    38.1 ms
읽기 : CSV    210.3 ms | Parquet    10.2 ms
크기 : CSV    27.91 MB | Parquet     6.48 MB
```

캡처 3. profile_perf.py - sys.getsizeof, timeit, cProfile 통합 측정

```
[ (.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 % python profile_perf.py ]
데이터 100,000행 생성

[sys / 메모리 크기]
sys.getsizeof(df) : 4,300,164 B
df.memory_usage(deep) : 4,300,132 B
list(range(N)) : 800,056 B (원소 전부 보유)
generator : 200 B (규칙만 보유)

[timeit / 시간 (ms)]
쓰기 CSV 44.0 | Parquet 13.8
읽기 CSV 18.9 | Parquet 10.6

[cProfile / 함수별 누적시간 상위 8]
4885 function calls (4826 primitive calls) in 0.072 seconds
Ordered by: cumulative time
List reduced from 602 to 8 due to restriction <8>
ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
1 0.000 0.000 0.072 0.072 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/profile_perf.py:41(save_and_read)
1 0.000 0.000 0.045 0.045 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/core/generic.py:3795(to_csv)
1 0.000 0.000 0.045 0.045 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/formats/format.py:976(to_csv)
1 0.000 0.000 0.045 0.045 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/formats/csvs.py:246(save)
1 0.000 0.000 0.045 0.045 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/formats/csvs.py:274(_save)
1 0.001 0.001 0.045 0.045 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/formats/csvs.py:307(_save_body)
4 0.029 0.007 0.044 0.011 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/formats/csvs.py:317(_save_chunk)
1 0.000 0.000 0.019 0.019 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/parsers/readers.py:349(read_csv)
[ (.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 % ]
```

캡처 4. python -m memory_profiler - save_and_read 함수의 라인별 메모리 증감

```
(.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 % python -m memory_profiler profile_perf.py
/Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/memory_profiler.py:752: DeprecationWarning: 'asyncio.iscoroutinefunction' is deprecated and slated for removal in Python 3.16; use inspect.iscoroutinefunction() instead
  if iscoroutinefunction(func):
데이터 100,000행 생성

[sys / 메모리 크기]
sys.getsizeof(df)      : 4,300,164 B
df.memory_usage(deep)  : 4,300,132 B
list(range(N))         : 800,056 B (원소 전부 보유)
generator              : 200 B (규칙만 보유)

[timeit / 시간 (ms)]
쓰기 CSV 43.3 | Parquet 8.9
읽기 CSV 19.2 | Parquet 6.4

[cProfile / 함수별 누적시간 상위 8]
4903 function calls (4844 primitive calls) in 0.079 seconds
Ordered by: cumulative time
List reduced from 614 to 8 due to restriction <8>
ncalls  tottime  percall  cumtime  percall filename:lineno(function)
  1      0.000      0.000      0.079      0.079 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/memory_profiler.py:759(f)
  1      0.000      0.000      0.079      0.079 profile_perf.py:41(save_and_read)
  1      0.000      0.000      0.045      0.045 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/core/generic.py:3795(to_csv)
  1      0.000      0.000      0.046      0.046 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/formats/format.py:1976(to_csv)
  1      0.000      0.000      0.046      0.046 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/formats/csvs.py:246(save)
  1      0.000      0.000      0.045      0.045 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/formats/csvs.py:274(_save)
  1      0.001      0.001      0.045      0.045 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/formats/csvs.py:307(_save_body)
  4      0.029      0.007      0.044      0.011 /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/lib/python3.14/site-packages/pandas/io/formats/csvs.py:317(_save_chunk)
Filename: profile_perf.py

Line #    Mem usage    Increment  Occurrences   Line Contents
=====
  41  177.281 MiB  177.281 MiB          1   @profile
  42
  43      def save_and_read(df):
  44          # memory_profiler를 실행하면 이 함수의 라인별 메모리 증감을 보여준다
  45          OUT_DIR.mkdir(parents=True, exist_ok=True)
  46          csv_path = OUT_DIR / "p.csv"
  47          pq_path = OUT_DIR / "p.parquet"
  48          df.to_csv(csv_path, index=False)
  49          df.to_parquet(pq_path, index=False)
  50          csv_back = pd.read_csv(csv_path)
  51          pq_back = pd.read_parquet(pq_path)
  52          return csv_back, pq_back
```

캡처 5. pytest -v - 스키마 검증 테스트 6건 전부 통과

```
(.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 % pytest -v
===== test session starts =====
platform darwin -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 -- /Users/kwonkyunghyun/workspace/skala-data-project-mission1/.venv/bin/python3.14
cachedir: .pytest_cache
rootdir: /Users/kwonkyunghyun/workspace/skala-data-project-mission1
plugins: anyio-4.14.2
collected 6 items

test_main.py::test_valid_weather_passes PASSED [ 16%]
test_main.py::test_precipitation_out_of_range_fails PASSED [ 33%]
test_main.py::test_temperature_type_error PASSED [ 50%]
test_main.py::test_ip_coordinate_range[-91-0] PASSED [ 66%]
test_main.py::test_ip_coordinate_range[0-200] PASSED [ 83%]
test_main.py::test_validate_splits_valid_and_errors PASSED [100%]

===== 6 passed in 11.79s =====
(.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 %
```

캡처 6. ruff check - 코드 스타일 검사 통과

```
(.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 % ruff check .
All checks passed!
(.venv) kwonkyunghyun@gwongyeonghyeon-ui-MacBookPro skala-data-project-mission1 % ruff check --fix .
All checks passed!
```